CodeSport

Sandya Mannarswamy

Welcome back to 'CodeSport'. In this month's column, we discuss mechanisms for design patterns that can help simplify object-oriented programming and facilitate software reuse.

Many readers had sent their answers to the questions we featured in the January edition. Congratulations to our readers K. Madhu and Joseph Raj for their correct solutions to all the questions.

I have received requests from a few readers to discuss software reuse and design patterns, so over the next few months, we will focus on these topics. Presently, we will look at how design patterns can help software reuse. We assume that the reader is familiar with basic object-oriented programming concepts.

Software reuse

The importance of software reuse has been widely understood and is a major advantage of object-oriented programming. There are two possible ways of reusing functionality in object-oriented programming. One is through class inheritance and the other is through object composition.

Reuse by class inheritance

Using class inheritance, we can define the implementation of one class in terms of another. Consider the often quoted example of a class hierarchy that defines a base class called 'Employee' and a derived class 'Manager', which is derived from this base class. By deriving the 'Manager' class from the 'Employee' class, we do not have to rewrite the employee functionality needed in the Manager class, since a Manager is also an Employee. Only newer functionality pertaining to the management function of the Manager class needs to be defined and implemented. This form of reuse is known as 'White box reuse', since typically, when reused through class inheritance, the internals of the base class are visible/exposed to the derived classes.

While reuse by class inheritance is easy, it has some disadvantages as well. Since class inheritance is defined at compile time, the implementations inherited from the base classes can not be changed dynamically at runtime. The second major disadvantage is that sub-classing often results in exposing the base class implementation to the derived class. In many real-life cases, the implementation of a derived class is so closely tied up with the implementation details of the base class, that any change in the base class will impact the derived class as well.

Consider the following class representation where we have the Employee class which represents an employee, and a Manager class that inherits from it. Let us assume that the dates are represented as dd-mm-yyyy in the Employee class, and we have a method in the Employee class, namely PrintDateOfJoining, which prints the date of joining in DD-MM-YY format. Now what if the Manager class was designed so that it has a *Print* method that prints the Manager details as given in the code snippet below:

```
void Manager::Print()
{
        printf("employee id: %d  name: %s \n", this->id, this->name)
        printf ("date of joining (dd-mm-yy) is:\n);
        this->PrintDateofJoining();
}
```

In this snippet, the *Print* method in the derived Manager class calls the PrintDateOfJoining method. If this method was redesigned to print the date as DD-MM-YYYY format, the *Print* method needs to change the printing of the title message as:

```
void Manager::Print()
{
        printf("Manager id: %d  name: %s \n", this->id, this->name)
        printf ("date of joining (dd-mm-yyyy) is:\n);
        this->PrintDateofJoining();
}
```

This is a bad design since the base class implementation of printing dates as DD-MM-YY was exposed to the derived class. This dependence meant that the derived class needed to change when there was a change in the base class implementation of printing the date of joining. The problem occurred because the base class implementation was inherited assuming a certain underlying representation. When the underlying representation in the base class changed, that required a corresponding change in the derived class, which is undesirable. A simple code change could have avoided the dependency as shown below:

```
void Manager::Print()
{
        printf("Manager id: %d  name: %s \n", this->id, this->name)
        printf ("date of joining (%) is:\n",
```